

Artifact Appendix for KubePACS: Kubernetes Cluster Using Performant, Highly Available, and Cost Efficient Spot Instances

Taeyoon Kim*
tykim7@hanyang.ac.kr
Dept. of Data Science
Hanyang University
Seoul, Republic of Korea

Kyumin Kim*
okkimok123@hanyang.ac.kr
Dept. of Data Science
Hanyang University
Seoul, Republic of Korea

Enrique Molina-Giménez
enrique.molina@urv.cat
Dept. of Computer Eng. and Math.
Universitat Rovira i Virgili
Tarragona, Spain

Pedro García-López
pedro.garcia@urv.cat
Dept. of Computer Eng. and Math.
Universitat Rovira i Virgili
Tarragona, Spain

Kyungyong Lee†
kyungyong@hanyang.ac.kr
Dept. of Data Science
Hanyang University
Seoul, Republic of Korea

Abstract

This artifact accompanies KubePACS, a Kubernetes-native spot-instance provisioning system that jointly considers hardware performance, spot pricing, and multi-node availability. It provides the optimizer, a modified Karpenter controller, deployment configurations, stored experimental results, and scripts for generating the paper’s figures and tables. To minimize cloud cost and setup effort, the primary evaluation workflow processes the supplied results locally without an AWS account. Separate API and Kubernetes workflows support functional evaluation of the implementation. We request the *Artifacts Available*, *Artifacts Functional*, and *Results Reproduced* badges.

1 Artifact Check-List

- **Programs:** Python optimizer and analysis scripts; Go-based Karpenter integration; Terraform and Helm deployment configuration.
- **Software:** Python 3.11 and uv for figures, using the supplied lock file. Table generation uses the Python standard library.
- **Run-time environment:** Linux workstation for local analysis; AWS credentials for API checks; AWS EKS for optional node provisioning.
- **Hardware:** General-purpose CPU; no GPU required for local result generation. Controller container builds target Linux AMD64.
- **Disk space:** Approximately 4 GiB for the local figure workflow, including dependencies, temporary inputs, and generated outputs.
- **Execution time:** Allow several minutes for all figures; each plotting script has a 180-second timeout.
- **Outputs:** 24 figure PDFs; Tables 2 and 3 in CSV, Markdown, and LaTeX; optimizer allocations and controller logs for functional checks.
- **Access:** Source, data, and instructions at <https://github.com/ddps-lab/KubePACS>.

2 Description

2.1 Access and Structure

The repository is the entry point for installation and evaluation. Commands in this appendix start at its root.

```
KubePACS/  
|-- README.md  
|-- figures/  
|   |-- reproduce.py  
|   |-- figure*/  
|   |-- table2_alpha/  
|   +-- table3_compute/  
|-- api/  
|-- KubePACS_with_Karpenter/  
|-- IaC/IaC_karpenter_kubepacs/  
|-- charts/  
+-- frontend/
```

2.2 Software and Data

Optimizer and integration. The optimizer combines integer linear programming with Golden Section Search to select instance types, availability zones, and node counts. The `api/` directory exposes this implementation through a Lambda handler that can also be invoked locally. The `KubePACS_with_Karpenter/` directory contains the modified Karpenter core and AWS provider. Terraform and Helm files provide infrastructure and controller deployment workflows. The website is optional for evaluation.

Experimental inputs. The `figures/` directory contains benchmark data, stored simulation and application results, plotting scripts, and reference PDFs. Historical price inputs are supplied locally, so figure generation does not query live market data. Table 2 aggregates allocation and alpha-sweep results, including the supplied $\alpha=1$ records. Table 3 uses the recorded compilation and video encoding throughput and instance prices. Figures 3 and 4 are architectural illustrations supplied as PPTX sources; Table 1 defines mathematical notation.

3 Relationship to the Paper

The artifact connects the paper’s claims to implementations and stored results as follows. Figure and table numbers below refer to the main paper.

*Both authors contributed equally to this work.

†Corresponding author.

- **Cost-performance optimization:** benchmark and price comparisons and baseline evaluations in Figures 1, 5, and 10.
- **Multi-node availability:** recorded request fulfillment and availability indicators in Figures 2 and 9.
- **Adaptive optimization:** alpha sensitivity, search overhead, and configuration comparisons in Figures 6–7 and Table 2.
- **Workload-aware selection:** specialized instance selection in Figure 8 and compute-workload results in Figure 11 and Table 3.
- **Kubernetes integration:** the modified controller and deployment configuration support a separate provisioning check.

Figure 12 additionally plots stored fault-injection outcomes. Its plotting script uses the supplied data; live AWS Fault Injection Service experiments are not part of the evaluation workflow described here.

4 Installation and Execution

4.1 Local Figure and Table Generation

Install the locked analysis environment and run:

```
uv sync --locked --project figures
uv run --locked --project figures \
  python figures/reproduce.py
python3 figures/table2_alpha/table_02.py
python3 figures/table3_compute/table_03.py
```

Dependency installation requires Internet access; these generation commands then operate on local inputs. No cluster or AWS credentials are required. Choose a new `--output` directory for subsequent runs.

Data processing and collection. The figure runner executes 16 scripts in a temporary input copy, leaving reference files unchanged. It collects PDFs, per-script logs, and `summary.json` under `artifact-results/figures/`. Table outputs are collected under `artifact-results/table2/` and `artifact-results/table3/`. The table commands emit `table2.csv/.md/.tex` and `table3.csv/.md/.tex`; Table 2 also emits `table2_details.csv` with 1,200 normalized observations.

Expected outputs. The figure command exits with code 0 and reports PASS: 16 scripts, 24 PDFs. The summary lists script exit codes and output-presence checks. The table commands report PASS: Table 2 and PASS: Table 3. Table 2 checks all five rounded values against the paper: 0.8616, 0.9563, 0.0006, 0.0001, and 1.0000 for Greedy, $\alpha=0$, $\alpha=0.5$, $\alpha=1$, and GSS, respectively.

Interpretation. Compare generated plots with the supplied references using the metric definitions and figure-by-figure mapping in `figures/README.md`. Table 2 normalizes each configuration's efficiency to GSS for the same run and scenario, then averages 240 ratios per configuration. Table 3 summarizes the two compute workloads and their cost and throughput comparisons. The local workflow reprocesses stored experiments; API and EKS execution exercise the software separately.

4.2 Optimizer and Kubernetes Evaluation

API. Follow `api/README.md` to install the API dependencies and invoke `lambda_handler` with pod count, per-pod CPU and memory, and region. The default market input is `api/data/latest_aws.json`. AWS metadata access requires `ec2:DescribeInstanceTypes` and `ec2:DescribeAvailabilityZones`. A successful request returns status 200 and a nonempty allocation containing instance

type, availability zone, and positive instance count. This check does not provision nodes.

EKS. Use the build and deployment instructions in `KubePACS_with_Karpenter/README.md` and `IaC/IaC_karpenter_kubepacs/README.md`. This path requires Docker, AWS CLI, `kubectl`, Helm 3, Terraform when creating infrastructure, appropriate IAM permissions, and available AWS capacity. A local market snapshot can be packaged or mounted in the controller using `KUBEPACS_SPOT_DATA_PATH`. Functional success requires a solver invocation, a usable allocation, corresponding `NodeClaims` and nodes, and ready workload pods without fallback to ordinary Karpenter. Collect controller logs, node labels, and workload events to inspect this sequence.

5 Notes

AWS deployment is optional and incurs charges. Review resource limits and the target account before deployment; follow the component READMEs for cleanup, removing test nodes while the controller is still running before tearing down the cluster. For local analysis, remove only the selected output directory when finished. Retain the evaluated source revision alongside outputs; development versions may change. Questions can be submitted through the repository's issue tracker.